

实验报告

曹诗竹 25300160053

一、在终端中执行 `./check_ops.py bits.c` 后的截图

```
cecilia@LAPTOP-F8EP5J01:/mnt/e/unniversity/term2/computersystem/lab1/25300160053-Lab1$ python3 check_ops.py bits.c
signMask: 1/2 operators
bitXor: 7/8 operators
negativePart: 4/6 operators
copyByteWithin: 9/12 operators
logicalShift: 6/20 operators
swapNibblePairs: 10/18 operators
secondLowestZeroBit: 7/8 operators
oddParity: 19/56 operators
rotateRightBits: 5/16 operators
roundEvenPow2: 20/24 operators
midpointTowardFirst: 22/32 operators
isBetweenEitherOrder: 47/48 operators
mul5Sat: 28/30 operators
classifyAdd3: 49/52 operators
floatScaleThreeHalves: 53/60 operators
floatRoundEven: 27/65 operators
float_i2f: 33/40 operators
bitCount: 28/40 operators
bitReverse: 33/34 operators
All 19 functions passed operator checks.
```

二、在终端中执行 `./btest` 后的截图

```
cecilia@LAPTOP-F8EP5J01:/mnt/e/unniversity/term2/computersystem/lab1/25300160053-Lab1$ ./btest
Score Rating Errors Function
1 1 0 signMask
2 2 0 bitXor
3 3 0 negativePart
4 4 0 copyByteWithin
4 4 0 logicalShift
4 4 0 swapNibblePairs
4 4 0 secondLowestZeroBit
5 5 0 oddParity
5 5 0 rotateRightBits
5 5 0 roundEvenPow2
5 5 0 midpointTowardFirst
7 7 0 isBetweenEitherOrder
7 7 0 mul5Sat
7 7 0 classifyAdd3
7 7 0 floatScaleThreeHalves
10 10 0 floatRoundEven
10 10 0 float_i2f
10 10 0 bitCount
10 10 0 bitReverse
Total points: 110/110
```

三、实现每个函数的思路

1、signMask:

```
int signMask(void) {
    return 1<<31;
}
```

目标是：得到最高位为 1、其余位为 0 的掩码，也就是符号位的掩码。实现过程如下：常数 1 的二进制是：0000 0000 0000 0000 0000 0000 0000 0001 它只有最低位，也就是第 0 位是 1。使用左移操作 `<<` 将它左移 31 位。左移后，原来的第 0 位移动到第 31 位，也就是最高位：1000 0000 0000 0000 0000 0000 0000 0000 这个结果正好就是 0x80000000。因此代码直接返回 `1<<31`，就得到了最高位为 1、其余位为 0 的掩码，也就是符号位的掩码。

2、bitXor:

```
int bitXor(int x, int y) {
    return ~(~x&~y)&~(x&y);
}
```

目标是只用 $\sim$ 和 $\&$ 两个运算符实现按位异或 $x \wedge y$ 。实现过程如下：我们知道，异或可以表示为： $x \wedge y = (x \mid y) \& \sim(x \& y)$ ，即“至少一个为 1，且不是两个都为 1”，即恰好一个为 1，这正是异或。但题目只允许使用 $\sim$ 和 $\&$ ，不允许使用 $\mid$ 。因此需要用德摩根律把 $\mid$ 转换成 $\sim$ 和 $\&$ ： $x \mid y = \sim(\sim x \& \sim y)$ ，因此得到： $x \wedge y = \sim(\sim x \& \sim y) \& \sim(x \& y)$ 。

3、negativePart:

```
int negativePart(int x){
    return (~x+1)&(x>>31);
}
```

目标是：如果 $x < 0$ ，返回 $-x$ ；否则返回 0。实现分为三个部分：首先是计算 $-x$ ，在补码表示中，对一个数取相反数的方法是“按位取反再加一”，即 $\sim x + 1$ ；其次是构造条件掩码，需要判断 x 是否为负数，整数 x 的最高位（第 31 位）是符号位，通过算术右移 31 位可以将符号位扩展到整个字， $x < 0$ 时 $x \gg 31$ 得到全 1， $x \geq 0$ 时 $x \gg 31$ 得到全 0，这个结果就充当了一个“掩码”：负数时全 1，非负数时全 0；最后，将 $-x$ 与掩码进行按位与，就能够实现 x 为负时返回其相反数，否则返回 0。

4、copyByteWithin:

```
int copyByteWithin(int x, int src, int dst) {
    int src_shift=src<<3;
    int dst_shift=dst<<3;
    int src_val=(x>>src_shift)&0xFF;
    int mask=~(0xFF<<dst_shift);
    return (x&mask)|(src_val<<dst_shift);
}
```

目标是：把 x 中第 src 个字节（从最低有效字节 0 开始编号）复制到第 dst 个字节，其他字节保持不变。 $src_shift = src \ll 3$ 和 $dst_shift = dst \ll 3$ 分别是第 src 个字节需要右移的位数和第 dst 个字节需要左移的位数。实现步骤如下：

第一，提取源字节， $x \gg src_shift$ 将第 src 个字节移动到最低 8 位， $\& 0xFF$ 屏蔽高 24 位，只保留最低 8 位，得到该字节的值；

第二，构造目标字节的清除掩码， $0xFF \ll dst_shift$ 将 $0xFF$ （低 8 位全 1）左移到第 dst 个字节的位置，得到一个在该字节位置全 1、其他位全 0 的数。按位取反 $\sim$ 后，该字节位置变为全 0，其他位变为全 1。因此 $x \& mask$ 会把 x 中第 dst 个字节清零，其他字节保持不变；

第三，将源字节放到目标位置， $src_val \ll dst_shift$ 把提取出的源字节左移到第 dst 个字节的位置；最后，合并结果，按位或 $\mid$ 将两者合并，目标字节被替换为源字节，其他字节不变。

5、logicalShift:

```
int logicalShift(int x, int n) {
    int mask = ~(((1 << 31) >> n) << 1);
    return (x >> n) & mask;
}
```

目标是实现逻辑右移：将 x 右移 n 位，高位补 0，而不是像有符号数算术右移那样高位补符号位。实现步骤如下： $1 \ll 31$ 生成一个只有最高位（第 31 位）为 1 的数，即 $0x80000000$ 。这是一个负数（最高位为符号位），对这个负数进行算术右移 n 位，由于符号位为 1，右移时高位会补 1。结果的高 n 位全为 1，并且原来的第 31 位的 1 被移动到第 $31-n$ 位，因此第 $31-n$ 位也是 1。将上一步的结果左移 1 位。左移时低位补 0，原来第 $31-n$ 位的 1 被移出（或移到更高位），最终得到一个高 n 位全为 1，其余位全为 0 的数。最后按位取反，得到高 n 位为 0、低 $32-n$ 位为 1 的掩码。将 x 算术右移 n 位后与掩码进行按位与，得到逻辑右移的结果。

6、swapNibblePairs:

```
int swapNibblePairs(int x) {
    int mask_low = 0x0F | (0x0F << 8);
    mask_low = mask_low | (mask_low << 16);
    int mask_high = 0xF0 | (0xF0 << 8);
    mask_high = mask_high | (mask_high << 16);
    int low_nibble = x & mask_low;
    int high_nibble = x & mask_high;

    return (low_nibble << 4) | ((high_nibble >> 4) & mask_low);
}
```

目标是交换 x 中每个字节（8 位）内部的高 4 位和低 4 位。实现步骤如下：

第一，构造掩码， $mask_low$ 用于选中每个字节的低 4 位， $0x0F$ 是低 4 位为 1，高 4 位为 0， $0x0F \ll 8$ 把 4 个 1 移动到第 1 个字节的低 4 位，两者按位或，得到第 0、1 字节的低 4 位掩码，再左移 16 位并或上自身，将这种模式复制到高 16 位，最终形成每个字节的低 4 位都是 1 的掩码，用相同的方式构造每个字节的高 4 位都是 1 的掩码 $mask_high$ 。

第二，使用掩码提取 x 中每个字节的低 4 位与高 4 位。

第三，交换位置，低字节左移 4 位变成高字节，高字节右移 4 位变成低字节，但要注意算术右移可能会产生多余的 1 污染结果，因此需要用 $mask_low$ 清除符号位的扩展。

第四，合并结果，位或运算合并交换位置后的低字节和高字节，得到每个字节高低 4 位交换后的结果。

7、secondLowestZeroBit:

```
int secondLowestZeroBit(int x) {
    int lowest_zero = ~x & (x + 1);
    int x_no_lowest_zero = x | lowest_zero;
    return ~x_no_lowest_zero & (x_no_lowest_zero + 1);
}
```

目标是返回一个掩码，该掩码只在 x 中从低位起第二个为 0 的位上置 1。如果不存在第二个 0 位（例如 x 全为 1），则返回 0。实现步骤如下：

第一，找到第一个 0 位，并将它变成 1，通过表达式 $\sim x \& (x + 1)$ 隔离出 x 中最低的 0 位，当 x 加上 1 时，最低的 0 位会变为 1，而它后面所有原本为 1 的位都会因进位变为 0， $\sim x$ 将 x 按位取反，因此最低的 0 位在 $\sim x$ 中变为 1。将 $\sim x$ 与 $x + 1$ 进行按位与，结果只有最低的 0 位为 1，其余位为 0。

第二，再次使用同样的操作找到新的最低 0 位，此时找到的最低 0 位正是原 x 中的第二个 0 位，返回的掩码中只有该位为 1。关于边界情况，如果 x 全是 1，或者只有 1 个 0，

根据如上流程都返回 0，表示不存在第二个 0 位。

8、oddParity:

```
int oddParity(int x) {
    int mask16 = 0xFF | (0xFF << 8);
    x ^= (x >> 16) & mask16;
    x ^= (x >> 8) & 0xFF;
    x ^= (x >> 4) & 0x0F;
    x ^= (x >> 2) & 0x03;
    x ^= (x >> 1) & 0x01;
    return !(x & 1);
}
```

目标是返回 x 的奇校验位：当 x 的二进制表示中 1 的个数为偶数时返回 1，为奇数时返回 0。实现步骤如下：

第一，折叠异或，通过 5 次异或操作计算所有位的异或和，首先将 x 的高 16 位右移到低 16 位，用 mask16 只保留低 16 位以忽略算术右移可能带来的高位符号扩展，然后与原来的 x 异或，此时 x 的低 16 位 = 原低 16 位 ^ 原高 16 位，随后依次将低 16 位中的高低 8 位异或并保留低 8 位，低 8 位中的高低 4 位异或并保留低 4 位，低 4 位中的高低 2 位异或并保留低 2 位，低 2 位中的高低 1 位异或并保留最低一位，此时 x 的最低位就是所有 32 位的异或和。根据异或的规律，若 x 中有奇数个 1，最终最低位为 1，若 x 中有偶数个 1，最终最低位为 0。

第二，取反得到奇校验位，返回 $!(x \& 1)$ ，若最低位为 1，即有奇数个 1，返回 0，若最低位为 0，即有偶数个 1，返回 1。

9、rotateRightBits:

```
int rotateRightBits(int x, int n) {
    n = n & 31;
    int shift = 32 + ~n + 1;
    int s = shift & 31;
    int mask = ~( (~0) << s );
    return ((x >> n) & mask) | (x << s);
}
```

目标是：将 x 的二进制位向右循环移动 n 位，即从右边移出的位补到左边。实现思路如下：

第一，限制 n 的范围，将 n 对 32 取模，得到 0..31 之间的值，防止左移 32 位这种未定义行为的出现。

第二，计算互补移位数 $s = (32 - n) \% 32$ ， $\sim n + 1$ 是 $-n$ 的补码表示，所以 $32 + \sim n + 1 = 32 - n$ 。当 $n = 0$ 时， $\text{shift} = 32$ ， $s = 32 \& 31 = 0$ 。当 $n > 0$ 时， $32 - n$ 在 1..31 之间， $s = 32 - n$ 。因此 s 就是左移的位数，且始终在 0..31 之间，避免了左移 32 位的未定义行为。

第三，构造掩码，清除算术右移的符号扩展， ~ 0 是全 1 (0xFFFFFFFF)。(~ 0) << s 将全 1 左移 s 位，低 s 位补 0，高 $32 - s$ 位为 1。取反 $\sim$ 后，得到一个低 s 位为 1、其余位为 0 的掩码。这个掩码的低 s 位为 1，恰好对应右移后需要保留的低 $32 - n$ 位（因为 $s = (32 - n) \% 32$ ）。当 $n > 0$ 时， $s = 32 - n$ ，所以掩码低 $32 - n$ 位为 1，高 n 位为 0。

第四，合并结果，将 x 右移 n 位。由于 x 可能为负数，C 语言中这是算术右移，高位会补符号位（全 1 或全 0）。但我们只需要低 $32 - n$ 位，所以用 $\& \text{mask}$ 将高位（包括符号扩展位）清零，只保留低 $32 - n$ 位；将 x 左移 s 位，当 $n = 0$ 时， $s = 0$ ，左移 0 位

不变，当 $n > 0$ 时， $s = 32 - n$ ，左移 $32 - n$ 位会把原来的低 n 位移到高 n 位，低 $32 - n$ 位补 0。两者按位或 $|$ ，就把右移得到的低位部分和左移得到的高位部分合并，形成完整的循环右移结果。

10、roundEvenPow2:

```
int roundEvenPow2(int x, int n) {
    int d=1<<n;
    int half=d>>1;
    int q=x>>n;
    int r=x&(d+~0);
    int r_is_half=!(r+~half+1);
    int r_bigger_than_half=(~((r+~half)>>31))&1;
    int q_is_odd=q&1;
    return (q+(r_bigger_than_half|r_is_half&q_is_odd))<<n;
}
```

目标是：将非负整数 x 舍入到最接近的 2^n 的倍数。如果 x 恰好位于两个相邻倍数正中间（即余数 r 等于 $\text{half} = 2^{n-1}$ ），则选择使商为偶数的那个倍数。实现步骤如下：

第一，计算间隔、半间隔、商和余数， d 是倍数间隔，即 2^n ， half 是半间隔，用于判断是否处于中间， q 是 x 除以 2^n 的商， r 是余数，这是因为 d 是 2 的幂， $d-1$ 的低 n 位全是 1，所以 $x \& (d-1)$ 就是 x 除以 d 的余数 $x \% 2^n$ 。

第二，判断余数和 half 的关系， $r_is_half = !(r + \sim\text{half} + 1)$ ， $\sim\text{half} + 1 = -\text{half}$ ，所以 $r + \sim\text{half} + 1 = r - \text{half}$ 。当 $r == \text{half}$ 时， $r - \text{half} = 0$ ，逻辑非 $!0 = 1$ ；否则为 0。因此 r_is_half 是一个布尔值，表示余数是否恰好等于半间隔。 $r_bigger_than_half = (\sim((r + \sim\text{half}) >> 31)) \& 1$ ， $\sim\text{half} = -\text{half} - 1$ ，所以 $r + \sim\text{half} = r - \text{half} - 1$ 。若 $r > \text{half}$ ，则 $r - \text{half} - 1 \geq 0$ ，其符号位为 0，算术右移 31 位得到 0，取反 $\sim 0 = -1$ ，再与 1 按位与得到 1。若 $r \leq \text{half}$ ，则 $r - \text{half} - 1 < 0$ ，符号位为 1，算术右移 31 位得到全 1（即 -1），取反得到 0，与 1 按位与得到 0。因此 $r_bigger_than_half$ 为 1 当且仅当余数大于半间隔。

第三，判断商的奇偶性， $q_is_odd = q \& 1$ ：取商的最低位，判断商是否为奇数，若为奇数，最低位是 1。

第四，决定是否进位，根据舍入规则：如果 $r > \text{half}$ ，无论商是奇是偶，都应该向上进位，即商加 1。如果 $r == \text{half}$ ，且商为奇数，则为了向偶数舍入，需要进位使商变为偶数；如果商为偶数，则不进位。如果 $r < \text{half}$ ，则直接舍去，不进位。

第五，计算最终结果，将调整后的商（原商加上可能的进位）左移 n 位，即乘以 2^n ，得到舍入后的倍数。

11、midpointTowardFirst

```
int midpointTowardFirst(int x, int y) {
    int mid=(x&y)+((x^y)>>1);
    int odd=(x^y)&1;
    int x_sign=x>>31;
    int y_sign=y>>31;
    int x_and_y_sign_different=x_sign^y_sign;
    int x_bigger_when_sign_different=!x_sign;
    int x_bigger_when_sign_same=(~((x+~y+1)>>31))&1;
    return mid+(odd&(x_and_y_sign_different&x_bigger_when_sign_different|(~x_and_y_sign_different)&x_bigger_when_sign_same));
}
```

目标是计算 $(x + y) / 2$ 的精确数学中点，且不发生溢出。如果 $x + y$ 是奇数，中点恰好位于两个整数之间，此时选择更接近第一个参数 x 的那个整数。实现步骤如下：

第一，无溢出计算平均值向下取整，利用恒等式： $x + y = (x \& y) * 2 + (x \wedge y)$ ，有 $(x + y) / 2 = (x \& y) + (x \wedge y) / 2$ ， $\text{mid} = (x \& y) + ((x \wedge y) >> 1)$ ，当 $x + y$ 为偶数时， $(x \wedge y)$ 最低位为 0， $(x$

$(x \oplus y) \gg 1$ 等于 $(x \oplus y) / 2$, mid 就是精确中点。当 $x + y$ 为奇数时, $(x \oplus y)$ 最低位为 1, $(x \oplus y) \gg 1$ 等于 $(x \oplus y - 1) / 2$ (向下取整), 因此 mid 等于 $\text{floor}((x + y) / 2)$, 即两个相邻整数中较小的那个。

第二, 判断和是否为奇数, $x + y$ 的奇偶性与 $x \oplus y$ 的最低位相同。 $\text{odd} = 1$ 表示 $x + y$ 为奇数, 中点位于两个整数之间, 需要决定舍入方向。

第三, 判断 x 和 y 的相对大小, 需要判断 $x > y$ 还是 $x < y$, 以便决定中点更接近谁。由于可能涉及负数, 不能直接比较, 代码通过符号位和减法来判断。当符号不同时, 非负数一定大于负数, 因此若 x 非负 ($x_sign == 0$), 则 $x > y$ 。 $x_bigger_when_sign_different = !x_sign$ 在 x 非负时为 1, 否则为 0。当符号相同时, 通过计算 $x - y$ 来判断。 $x + \sim y + 1$ 等于 $x - y$ 。若 $x \geq y$, $(x - y) \gg 31$ 为 0, 取反后为 -1, 与 1 按位与得 1; 若 $x < y$, $(x - y) \gg 31$ 为 -1, 取反后为 0, 与 1 按位与得 0, 因此 $x_bigger_when_sign_same$ 为 1 表示 $x \geq y$, 为 0 表示 $x < y$ 。综合两种情况, $x_bigger = 1$ 表示 $x > y$, $x_bigger = 0$ 表示 $x < y$ 。

第四, 决定是否加一, 当 $\text{odd} = 1$ 时, 中点为两个整数之间: 若 $x > y$, 中点更接近 x , 而 x 是较大的数, 所以应选择较大的整数 $\text{mid} + 1$ 。若 $x < y$, 中点更接近 y , 而 y 是较小的数, 所以应选择较小的整数 mid 。因此最终返回 $\text{mid} + (\text{odd} \& x_bigger)$, 当 $\text{odd} = 1$ 且 $x > y$ 时, $\text{odd} \& x_bigger = 1$, 返回 $\text{mid} + 1$; 否则返回 mid 。

12、isBetweenEitherOrder:

```
int isBetweenEitherOrder(int x, int a, int b) {
    int sx = x >> 31;
    int sa = a >> 31;
    int sb = b >> 31;
    int diff_xa = sx ^ sa;
    int diff_xb = sx ^ sb;

    int not_x = ~x;
    int x_minus_a = x + ~a + 1;
    int a_minus_x = a + not_x + 1;
    int x_minus_b = x + ~b + 1;
    int b_minus_x = b + not_x + 1;

    int x_minus_a_neg = x_minus_a >> 31;
    int a_minus_x_neg = a_minus_x >> 31;
    int x_minus_b_neg = x_minus_b >> 31;
    int b_minus_x_neg = b_minus_x >> 31;

    int ge_xa = (diff_xa & !sx) | (!diff_xa & !x_minus_a_neg);
    int ge_ax = (diff_xa & !sa) | (!diff_xa & !a_minus_x_neg);
    int ge_xb = (diff_xb & !sx) | (!diff_xb & !x_minus_b_neg);
    int ge_bx = (diff_xb & !sb) | (!diff_xb & !b_minus_x_neg);

    return (ge_xa & ge_bx) | (ge_xb & ge_ax);
}
```

目标是判断整数 x 是否位于由 a 和 b 构成的闭区间内, 且 a 、 b 的顺序可以任意。即判断以下两个条件是否至少有一个成立: $a \leq x \leq b$, $b \leq x \leq a$ 。由于整数比较可能因减法溢出而

得到错误结果，代码采用“符号位优先”的策略来安全地比较两个有符号整数的大小。实现步骤如下：

第一，提取符号位，对 32 位有符号整数进行算术右移 31 位， s_x 、 s_a 、 s_b 分别表示 x 、 a 、 b 的符号：0 表示非负，-1 表示负。

第二，判断符号是否相同，利用异或运算，符号相同时， $0 \wedge 0 = 0$ 或 $(-1) \wedge (-1) = 0$ ，结果为 0，符号不同时， $0 \wedge (-1) = -1$ ，结果为 -1（非零）。所以 $\text{diff_xa} == 0$ 表示 x 与 a 同号， $\text{diff_xa} != 0$ 表示异号。 diff_xb 同理。

第三，计算差值和符号，为了在符号相同时比较大小，需要计算差值， $\sim a + 1$ 是 $-a$ 的补码表示，所以 $x + \sim a + 1 = x - a$ 。同理其他差值，然后取差值的符号位，为 0 表示差值非负，为 -1 表示差值为负。

第四，判断大小关系，需要判断 $x \geq a$ 、 $a \geq x$ 、 $x \geq b$ 、 $b \geq x$ 。对于每一对，根据符号是否相同分两种情况：符号不同时，若 x 非负（ $s_x == 0$ ，即 $!s_x$ 为 1），则非负数一定大于负数，所以 $x \geq a$ 成立。若 x 为负（ $s_x == -1$ ， $!s_x$ 为 0），则 x 是负数， a 是非负数， $x \geq a$ 不成立。因此这一分支的结果为 $\text{diff_xa} \& !s_x$ 。注意 diff_xa 在异号时为 -1（所有位为 1），与 $!s_x$ 按位与后，若 $!s_x$ 为 1，结果非零（真）；若 $!s_x$ 为 0，结果为 0（假）。符号相同时，此时直接看 $x - a$ 的符号。若 $x - a \geq 0$ ，即 $x_minus_a_neg == 0$ ，则 $!x_minus_a_neg$ 为 1，表示 $x \geq a$ 。因此这一分支的结果为 $! \text{diff_xa} \& !x_minus_a_neg$ 。合并两种情况， ge_xa 为 1 表示 $x \geq a$ ，为 0 表示 $x < a$ ，其余变量同理。

第五，判断是否在区间内闭区间有两种可能： $a \leq x \leq b$ ：即 $x \geq a$ 且 $x \leq b$ ，对应 ge_xa 为真且 ge_bx 为真（因为 ge_bx 表示 $b \geq x$ ，即 $x \leq b$ ）。 $b \leq x \leq a$ ：即 $x \geq b$ 且 $x \leq a$ ，对应 ge_xb 为真且 ge_ax 为真。只要满足其中一种，就说明 x 在区间内。

13、mul5Sat:

```
int mul5Sat(int x) {
    int x4 = x << 2;
    int high = x >> 29;
    int high_is_0 = !high;
    int high_is_m1 = !~high;
    int x4_overflow = !(high_is_0 | high_is_m1);
    int y = x4 + x;
    int add_overflow = ((x4 ^ y) & (x ^ y)) >> 31;
    int overflow = x4_overflow | (add_overflow & 1);
    int sign = x >> 31;
    int max = ~(1 << 31);
    int min = 1 << 31;
    int sat = (sign & min) | (~sign & max);
    int mask = ~overflow + 1;
    return (y & ~mask) | (sat & mask);
}
```

目的是：计算 $x * 5$ ，若结果溢出 32 位有符号整数范围，则饱和到 INT_MAX （正溢出）或 INT_MIN （负溢出）。实现步骤如下：

第一，计算 $x*4$ ，即 $x4=x \ll 2$

第二，检测 $x*4$ 是否溢出，左移 2 位相当于乘以 4。若 x 的高 3 位（第 29~31 位）不全为 0 或全为 1，则左移会改变符号位导致溢出。

第三，计算 $x*5$ ，即 $y=x*4+x$

第四，检测 $x*5$ 是否溢出：若两个加数符号相同，而结果符号与它们不同，则溢出。

第五，综合溢出标志， $overflow = x4_overflow \mid (add_overflow \& 1); overflow = 1$ 表示最终结果溢出，需要饱和。

第六，确定饱和值，根据 x 的符号选择 INT_MAX 或 INT_MIN 。

第七，根据溢出标志选择返回 y 或饱和值，若未溢出， $mask = 0$ ，返回 y 。若溢出， $mask = -1$ ，返回 sat 。

14、classifyAdd3

:

```
int classifyAdd3(int x, int y, int z) {
    int s1 = x + y;
    int c1 = ((x & y) | ((x | y) & ~s1)) >> 31 & 1;
    int s2 = s1 + z;
    int c2 = ((s1 & z) | ((s1 | z) & ~s2)) >> 31 & 1;
    int sum_sign = (x >> 31) + (y >> 31) + (z >> 31);
    int H = c1 + c2 + sum_sign;
    int s2_sign = s2 >> 31;
    int h_ge_1 = !(H >> 31) & !!H;
    int h_is_0 = !H;
    int h_is_m1 = !(H ^ ~0);
    int h_is_m2 = !(H ^ ~1);
    int h_is_m3 = !(H ^ ~2);
    int h_le_m2 = h_is_m2 | h_is_m3;
    int s2_neg = s2_sign & 1;
    int s2_nonneg = !s2_neg;
    int cond1 = h_ge_1 | (h_is_0 & s2_neg);
    int cond_m1 = h_le_m2 | (h_is_m1 & s2_nonneg);
    return cond1 + ~cond_m1 + 1;
}
```

目的是：判断三个有符号整数 x 、 y 、 z 的精确数学和 $x+y+z$ 是否超出 32 位有符号整数范围。若大于 INT_MAX 返回 1，若小于 INT_MIN 返回 -1，否则返回 0。不能使用更宽整数类型，只能使用位运算和 32 位加法。实现步骤如下：

第一，分解加法，将三数之和分解为两次二元加法：先计算 $s1 = x + y$ ，再计算 $s2 = s1 + z$ ，每一步都可能发生有符号溢出

第二，计算每一步的无符号进位，对于每次加法，使用公式 $((a \& b) \mid ((a \mid b) \& \sim sum)) \gg 31 \& 1$ 计算从最高位向第 32 位的无符号进位， $c1$ 表示 $x+y$ 的无符号进位， $c2$ 表示 $s1+z$ 的无符号进位，无符号进位结合操作数的符号位，可以推断有符号溢出的方向。

第三，统计三个操作数的符号位之和，计算 $\text{sum_sign} = (x \gg 31) + (y \gg 31) + (z \gg 31)$ 。每个负数算术右移 31 位得到 -1，非负数得到 0。因此 sum_sign 等于负数个数的相反数，取值范围为 -3 到 0。

第四，计算综合指标 H，令 $H = c1 + c2 + \text{sum_sign}$ 。H 综合了两次加法的进位信息和原始操作数的符号信息，其取值范围为 -3 到 2，可以反映精确和的符号趋势和溢出方向。

第五，提取最终截断结果 s2 的符号， $\text{s2_sign} = \text{s2} \gg 31$ 得到 s2 的符号位， $\text{s2_neg} = \text{s2_sign} \& 1$ 得到 0 或 1，表示 s2 是否为负， $\text{s2_nonneg} = !\text{s2_neg}$ 表示 s2 是否非负。

第六，判断正负溢出条件 cond1 ，当 $H > 0$ ，或者 $H == 0$ 且 s2 为负时，精确和大于 INT_MAX ，当 $H \leq -2$ ，或者 $H == -1$ 且 s2 非负时，精确和小于 INT_MIN 。

第七，返回结果，最终返回 $\text{cond1} + \sim \text{cond_m1} + 1$ ，即 $\text{cond1} - \text{cond_m1}$ ，若 $\text{cond1} = 1$ 、 $\text{cond_m1} = 0$ ，返回 1（正溢出），若 $\text{cond1} = 0$ 、 $\text{cond_m1} = 1$ ，返回 -1（负溢出），若两者均为 0，返回 0（未溢出）。

15、floatScaleThreeHalves:

```
unsigned floatScaleThreeHalves(unsigned uf) {
    unsigned sign = uf & 0x80000000;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFFFF;
    if (exp == 0xFF) return uf;
    if (exp == 0 && frac == 0) return sign;
    if (exp == 0) {
        unsigned N = frac * 3;
        if (N >= 0x1000000) {
            unsigned new_frac = (N >> 1) & 0x7FFFFFFF;
            if ((N & 1) && (new_frac & 1)) {
                new_frac++;
                if (new_frac == 0x800000)
                    return sign | 0x1000000;
            }
            return sign | 0x800000 | new_frac;
        } else {
            unsigned M_non = N >> 1;
            if ((N & 1) && (M_non & 1))
                M_non++;
            return sign | M_non;
        }
    } else {
        unsigned N = ((0x800000) | frac) * 3;
        unsigned new_exp, new_frac;

        if (N >= 0x2000000) {
            new_frac = (N >> 2) & 0x7FFFFFFF;
            new_exp = exp + 1;
            unsigned round_bits = N & 3;
            if (round_bits > 2 || (round_bits == 2 && (new_frac & 1))) {
                new_frac++;
                if (new_frac == 0x800000) {
                    new_frac = 0;
                    new_exp++;
                }
            }
        }
    }
}
```

```

    } else {
        new_frac = (N >> 1) & 0x7FFFFFFF;
        new_exp = exp;
        if (N & 1) {
            if (new_frac & 1) {
                new_frac++;
                if (new_frac == 0x800000) {
                    new_frac = 0;
                    new_exp++;
                }
            }
        }

        if (new_exp >= 0xFF)
            return sign | 0x7F800000;
        return sign | (new_exp << 23) | new_frac;
    }
}

```

目的是：计算单精度浮点数 $f * 3/2$ ，输入和输出均为 unsigned int 表示的单精度浮点位模式。要求使用 round-to-nearest-even 舍入，保留 +0 和 -0 的符号，当输入为 NaN 时返回原值。实现步骤如下：

第一，提取符号、指数和尾数，从 uf 中分离出符号位 sign、8 位指数 exp 和 23 位尾数 frac。

第二，处理特殊情况，若 $exp == 0xFF$ （即 NaN 或无穷大），直接返回 uf，若 $exp == 0$ 且 $frac == 0$ （即 ± 0 ），返回 sign，保留零的符号。

第三，处理非规格化数，即 $exp == 0$ 且 $frac != 0$ ，非规格化数的数值为 $frac * 2^{(-149)}$ 。乘以 $3/2$ 后，计算 $N = frac * 3$ ，然后根据 N 的大小决定结果是否需要规格化：若 $N \geq 0x1000000$ （即 $N \geq 2^{24}$ ），说明结果将变为规格化数，计算 $new_frac = (N >> 1) \& 0x7FFFFFFF$ ，并进行 round-to-nearest-even 舍入：若 N 为奇数且 new_frac 的最低位为 1，则 new_frac++ 。若舍入导致 $new_frac == 0x800000$ ，则返回 $sign | 0x1000000$ （指数为 1，尾数为 0）；否则返回 $sign | 0x800000 | new_frac$ （指数为 1，隐含的 1 加上尾数）。若 $N < 0x1000000$ ，结果仍为非规格化数。计算 $M_non = N >> 1$ ，同样进行 round-to-nearest-even 舍入：若 N 为奇数且 M_non 的最低位为 1，则 M_non++ 。返回 $sign | M_non$ 。

第四，处理规格化数，即 $exp != 0$ 且 $exp != 0xFF$ ，规格化数的有效数字为 $M = 0x800000 | frac$ （24 位，包含隐含的 1）。乘以 $3/2$ 后，计算 $N = M * 3$ 。根据 N 的大小决定结果是否需要调整指数：若 $N \geq 0x2000000$ （即 $N \geq 2^{25}$ ），说明结果的有效数字将超过 2.0，需要规格化并增加指数。计算 $new_frac = (N >> 2) \& 0x7FFFFFFF$ ， $new_exp = exp + 1$ ，舍入位为 $N \& 3$ 。进行 round-to-nearest-even 舍入：若舍入位大于 2，或等于 2 且 new_frac 最低位为 1，则 new_frac++ 。若 $new_frac == 0x800000$ ，则 $new_frac = 0$ ， new_exp++ 。若 $N < 0x2000000$ ，结果的有效数字在 [1.0, 2.0) 之间，指数不变。计算 $new_frac = (N >> 1) \& 0x7FFFFFFF$ ， $new_exp = exp$ ，舍入位为 $N \& 1$ 。进行 round-to-nearest-even 舍入：若 N 为奇

数且 new_frac 最低位为 1，则 new_frac++。若 new_frac == 0x800000，则 new_frac = 0，new_exp++。

第五，处理指数上溢，若调整后的 new_exp >= 0xFF，说明结果超出单精度浮点数范围，返回无穷大，sign | 0x7F800000

第六，组装结果，将符号位、新的指数位和新的尾数位组合。

16、floatRoundEven:

```
unsigned floatRoundEven(unsigned uf) {
    unsigned sign = uf & 0x80000000;
    unsigned exp = (uf >> 23) & 0xFF;
    unsigned frac = uf & 0x7FFFFFFF;
    if (exp == 0xFF) {
        return uf;
    }

    if (exp == 0) {
        return sign;
    }

    int E = exp - 127;
    if (E >= 23) {
        return uf;
    }
    if (E < -1) {
        return sign;
    }

    if (E == -1) {
        if (frac == 0) {
            return sign;
        } else {
            return sign | 0x3F800000;
        }
    }

    unsigned M = (1 << 23) | frac;
    int shift = 23 - E;
    unsigned I = M >> shift;
    unsigned frac_part = M & ((1 << shift) - 1);
    unsigned half = 1 << (shift - 1);

    if (frac_part > half || (frac_part == half && (I & 1))) {
        I++;
    }
    if (I == 0) {
        return sign;
    }
    int h = 31;
```

```

while ((I & (1 << h)) == 0) {
    h--;
}
unsigned exp_new = h + 127;
unsigned frac_new = (I << (23 - h)) & 0x7FFFFFFF;
return sign | (exp_new << 23) | frac_new;
}

```

目的是：将单精度浮点数 `uf` 舍入到最接近的整数，采用 `round-to-nearest-even`（四舍六入五成双）规则，返回该整数的单精度浮点位表示。若舍入结果为零，则保留输入符号（如负数舍入到零返回 `-0`）。若输入为 `NaN` 或无穷大，则原样返回。实现步骤如下：

第一，提取符号、指数和尾数，从 `uf` 中分离出符号位 `sign`、8 位指数 `exp` 和 23 位尾数 `frac`。

第二，处理 `NaN` 和无穷大，若 `exp == 0xFF`，说明是 `NaN` 或无穷大，直接返回 `uf`。

第三，处理非规格化数和零，若 `exp == 0`，其绝对值远小于 0.5，舍入到最近的整数为 0。直接返回 `sign`，保留零的符号。

第四，计算实际指数，对于规格化数，实际指数为 $E = \text{exp} - 127$ 。浮点数值为 $(1.\text{frac}) * 2^E$ 。

第五，处理已经是整数的情况，若 $E \geq 23$ ，数值的绝对值已经大于等于 2^{23} ，小数部分无法表示，本身就是整数，直接返回 `uf`。

第六，处理绝对值小于 0.5 的情况，若 $E < -1$ （即 $E \leq -2$ ），舍入到 0，返回 `sign`。

第七，处理 $E == -1$ 的情况，此时数值在 $[0.5, 1)$ 之间。若 `frac == 0`，值为 0.5，根据 `round-to-nearest-even` 舍入到偶数 0，返回 `sign`；若 `frac != 0`，值大于 0.5，舍入到 1，返回 `sign | 0x3F800000`（即 ± 1.0 的位表示）。

第八，处理一般规格化数，构造包含隐含 1 的 24 位有效数字 $M = (1 \ll 23) | \text{frac}$ ，需要右移 $\text{shift} = 23 - E$ 位来得到整数部分 $I = M \gg \text{shift}$ ，小数部分 $\text{frac_part} = M \& ((1 \ll \text{shift}) - 1)$ ，半位 $\text{half} = 1 \ll (\text{shift} - 1)$ 。

第九，进行舍入，若 $\text{frac_part} > \text{half}$ ，或者 $\text{frac_part} == \text{half}$ 且 I 为奇数（最低位为 1），则整数部分 $I++$ 。若 $I == 0$ ，说明舍入结果为 0，返回 `sign`。

第十，将整数 I 转换回单精度浮点格式，寻找 I 的最高有效位位置 h （从 31 向下扫描）。新的指数 $\text{exp_new} = h + 127$ 。新的尾数 $\text{frac_new} = (I \ll (23 - h)) \& 0x7FFFFFFF$ ，即把 I 的小数部分放入尾数位。最终返回 `sign | (exp_new << 23) | frac_new`。

17、float_i2f:

```

unsigned float_i2f(int x) {
    unsigned sign = 0;
    unsigned frac = 0;
    if (x == 0) return 0;
    unsigned ux;
    if (x < 0) {
        sign = 1 << 31;
        ux = -x;
    } else {
        ux = x;
    }
    int h = 31;
    while ((ux & (1 << h)) == 0) {
        h--;
    }
    unsigned exp = h + 127;
    if (h > 23) {
        unsigned lost = ux & ((1 << (h - 23)) - 1);
        frac = (ux >> (h - 23)) & ((1 << 23) - 1);
        unsigned half = 1 << (h - 23 - 1);
        if (lost > half) frac++;
        if (lost == half) {
            if ((frac & 1) != 0) frac++;
        }
        if (frac == (1 << 23)) {
            exp++;
            frac = 0;
        }
    } else {
        frac = (ux << (23 - h)) & 0x7FFFFFFF;
    }
    return sign | (exp << 23) | frac;
}

```

目的是: 将 32 位有符号整数 x 转换为单精度浮点数的位表示。返回值是 `unsigned int`, 但按 IEEE 754 单精度格式解释。要求使用 `round-to-nearest-even` 舍入规则。实现过程如下:

第一, 处理 0 值, 若 $x == 0$, 直接返回 0, 即 +0 的位表示。

第二, 提取符号并取绝对值, 若 $x < 0$, 设置符号位 $sign = 1 \ll 31$, 并将 x 取负得到无符号绝对值 $ux = -x$ 。若 $x \geq 0$, $sign = 0$, $ux = x$ 。这么操作是为了防止补码表示的负数扰乱后续操作。

第三, 寻找最高有效位, 从第 31 位向下扫描, 找到 ux 中最高位为 1 的位置 h 。 h 表示整数的最高有效位索引 (即 ux 的最高位是 2^h)。

第四, 计算指数, 规格化后, 浮点数的指数为 h , 加上偏置值 127, 得到 $exp = h + 127$ 。

第五, 根据有效位数决定位数处理的方式, 单精度浮点数尾数有 23 位, 加上隐含的 1 共 24 位有效数字。若 $h \leq 23$, 有效数字不超过 24 位, 无需舍入。将 ux 左移 $(23 - h)$ 位, 使最高位对齐到隐含的 1 位置, 然后取低 23 位作为尾数, 若 $h > 23$, 有效数字超过 24 位, 需要舍入: 计算需要右移的位数 $shift = h - 23$, 取出被移出的低位部分 $lost = ux \& ((1 \ll shift)$

-1), 将 ux 右移 $shift$ 位后, 取低 23 位作为初步尾数, 计算半位 $half = 1 \ll (shift - 1)$ 后进行舍入, 若 $lost > half$, 则 $frac++$, 若 $lost == half$, 则检查 $frac$ 的最低位: 若为 1 (奇数), 则 $frac++$, 使结果变为偶数, 若进位导致 $frac == (1 \ll 23)$, 说明尾数溢出, 需要将 $exp++$, 并将 $frac$ 置为 0。

第六, 组装最终结果。

18、bitCount:

```
int bitCount(int x) {
    int m1 = 0x55 | (0x55 << 8);
    m1 = m1 | (m1 << 16);
    int m2 = 0x33 | (0x33 << 8);
    m2 = m2 | (m2 << 16);
    int m4 = 0x0F | (0x0F << 8);
    m4 = m4 | (m4 << 16);
    int m8 = 0xFF | (0xFF << 16);
    int m16 = 0xFF | (0xFF << 8);
    x = (x & m1) + ((x >> 1) & m1);
    x = (x & m2) + ((x >> 2) & m2);
    x = (x & m4) + ((x >> 4) & m4);
    x = (x & m8) + ((x >> 8) & m8);
    x = (x & m16) + ((x >> 16) & m16);
    return x;
}
```

目的是: 统计 32 位整数 x 的二进制表示中 1 的个数 (即 `popcount`)。实现思路采用经典的“分治并行加法”算法, 将相邻位的 1 的个数逐层合并。每一步都将相邻的小组计数相加, 合并成更大的组, 由于每组位数足够容纳最大计数值 (例如 2 位最多存 2, 4 位最多存 4), 加法不会产生跨组进位。经过 5 次合并, 所有位的 1 的个数就汇总到了低 6 位中, 得到最终结果。

19、bitReverse:

```

int bitReverse(int x){
    int m1 = 0x55 | (0x55 << 8);
    m1 = m1 | (m1 << 16);
    int m2 = 0x33 | (0x33 << 8);
    m2 = m2 | (m2 << 16);
    int m4 = 0x0F | (0x0F << 8);
    m4 = m4 | (m4 << 16);
    int m8 = 0xFF | (0xFF << 16);
    int m16 = 0xFF | (0xFF << 8);

    x = ((x & m1) << 1) | ((x >> 1) & m1);
    x = ((x & m2) << 2) | ((x >> 2) & m2);
    x = ((x & m4) << 4) | ((x >> 4) & m4);
    x = ((x & m8) << 8) | ((x >> 8) & m8);
    x = ((x & m16) << 16) | ((x >> 16) & m16);
    return x;
}

```

目的是：将 32 位整数 x 的二进制位完全反转，即第 0 位与第 31 位交换，第 1 位与第 30 位交换，依此类推，返回反转后的整数。仍然采用分治交换法，从最小的位块开始逐层交换，最终完成整个 32 位的反转：使用五个掩码用于提取不同层次的位，分别移动奇数和偶数位后位或合并两者，以此依次交换相邻的 1 位、相邻的 2 位块、4 位块、8 位块、16 位块，最终使得所有位顺序颠倒。

四、对本实验的感受：

工作量略大。